someone else looks at the code later, it's clear that this was intended.

- *In order to force the compiler to generate the function when it would not otherwise have done so*—This is typically done with default constructors, which are only normally compiler generated if there are no user-defined constructors. If you need to define a custom copy constructor (for example), you can still get a compiler-generated default constructor by declaring it as defaulted.
- *In order to make a destructor virtual while leaving it as compiler generated.*
- *To force a particular declaration of the copy constructor, such as having it take the source parameter by a non-*`const` *reference rather than by a* `const` *reference.*
- *To take advantage of the special properties of the compiler-generated function, which are lost if you provide an implementation*—More on this in a moment.

Just as deleted functions are declared by following the declaration with `= delete`, defaulted functions are declared by following the declaration by `= default`, for example:

```
class Y
{
private:
    Y() = default;
public:
    Y(Y&) = default;
    T& operator=(const Y&) = default;
protected:
    virtual ~Y() = default;
};
```

Change access

Take a non-const reference

Declare as defaulted for documentation

Change access and add virtual

I mentioned previously that compiler-generated functions can have special properties that you can't get from a user-defined version. The biggest difference is that a compiler-generated function can be *trivial*. This has a few consequences, including the following:

- Objects with trivial copy constructors, trivial copy assignment operators, and trivial destructors can be copied with `memcpy` or `memmove`.
- Literal types used for `constexpr` functions (see section A.4) must have a trivial constructor, copy constructor, and destructor.